LAPORAN PRAKTIKUM PERTEMUAN 5



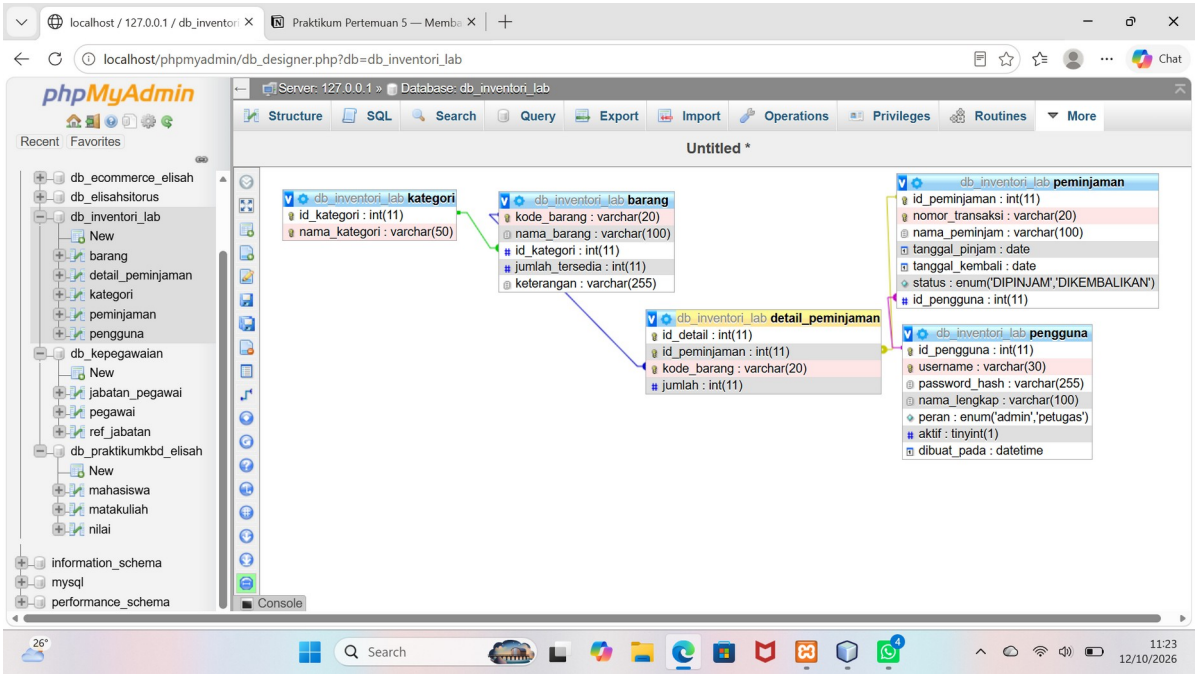
Dosen Pengampu: Juni Ismail, S.Kom., M.Kom.
Disusun Oleh :
Daniel Situmorang
25110105

A. Tujuan dan Gambaran Umum

Dalam praktikum ini dibuat dan diuji sebuah database inventori laboratorium bernama db_inventori_lab dengan MySQL melalui phpMyAdmin. Laporan berisi tangkapan layar yang disertai uraian tiap perintah SQL, diawali dengan struktur relasi antartabel, lalu perintah untuk membaca data (SELECT, WHERE, LIKE, JOIN, GROUP BY), perintah untuk memodifikasi data (INSERT, UPDATE, DELETE), dan ditutup dengan percobaan aturan integritas data (primary key, foreign key, NOT NULL, serta CHECK constraint) yang dirancang agar menghasilkan error.

1. Desain Database dan Relasi Antartabel (Designer)

Gambar berikut memperlihatkan tampilan Designer di phpMyAdmin untuk database db_inventori_lab, yang menyajikan semua tabel lengkap dengan kolom, tipe data, serta garis penghubung foreign key antartabel.



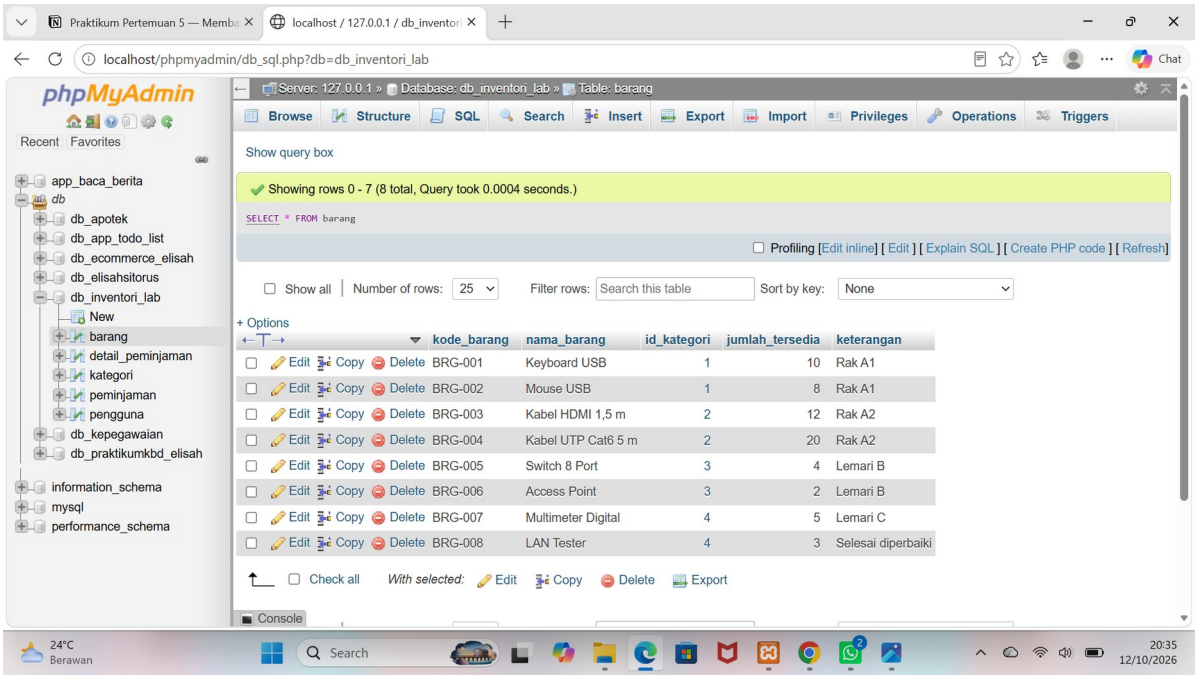
Hasil: Struktur database telah dinormalisasi. Data kategori, barang, petugas, dan transaksi disimpan terpisah pada tabel masing-masing, lalu dikaitkan lewat primary key dan foreign key sehingga konsistensi data terjaga.

2. Menampilkan Seluruh Data Barang

Query ini mengambil seluruh kolom dan seluruh baris dari tabel barang; simbol * mewakili "semua kolom".

Perintah SQL:

```
SELECT * FROM barang;
```



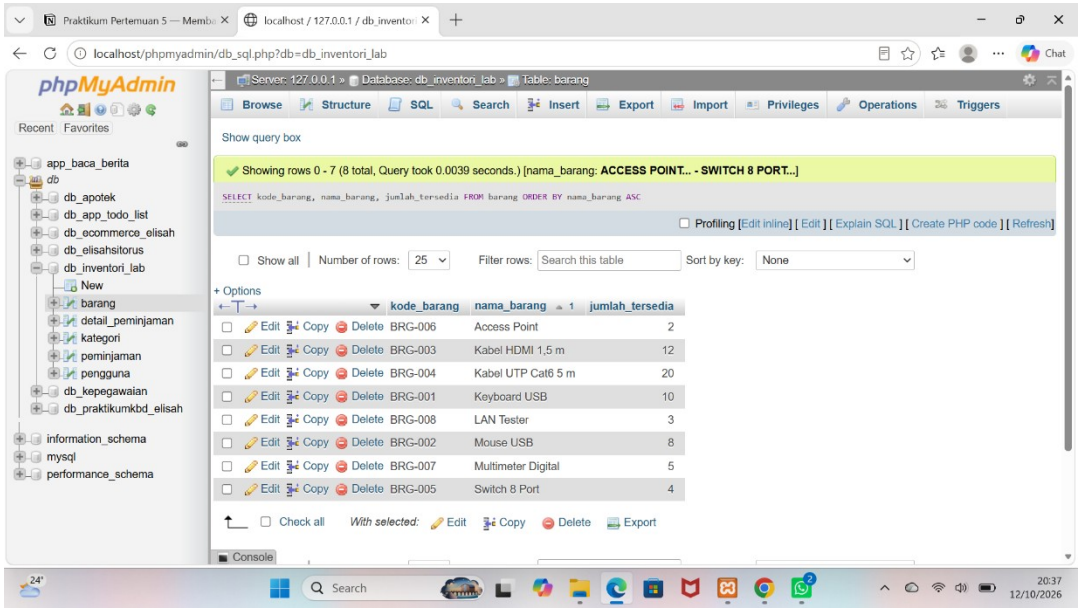
Hasil: SELECT * dipakai untuk melihat isi tabel secara lengkap sekaligus memeriksa bahwa data awal sudah tersimpan dengan benar.

3. Memilih Kolom Tertentu dan Mengurutkan Data

Query ini hanya memilih tiga kolom, yakni kode_barang, nama_barang, dan jumlah_tersedia, kemudian menyusun hasilnya menurut nama barang.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
ORDER BY nama_barang ASC;
```



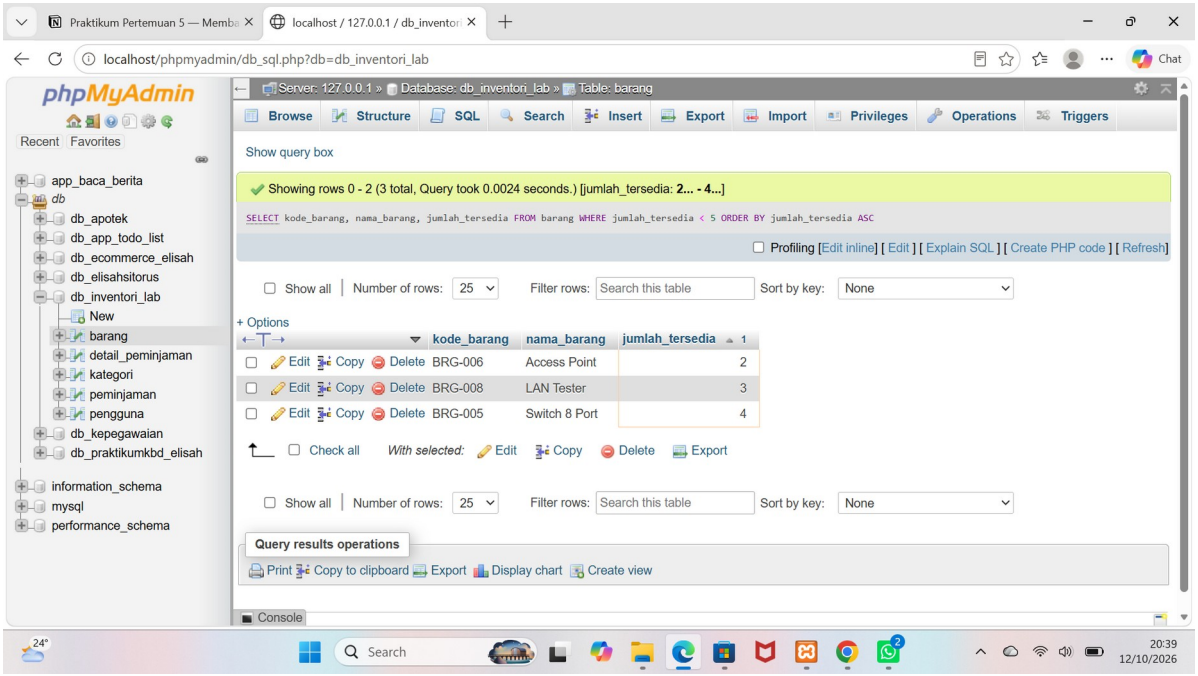
Hasil: Output berjumlah 8 baris dan terurut sesuai abjad nama barang.

4. Menyaring Data dengan WHERE (Stok Menipis)

Query ini menampilkan barang dengan stok yang hampir habis (jumlah tersedia di bawah 5), dimulai dari yang stoknya paling sedikit.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
WHERE jumlah_tersedia < 5
ORDER BY jumlah_tersedia ASC;
```



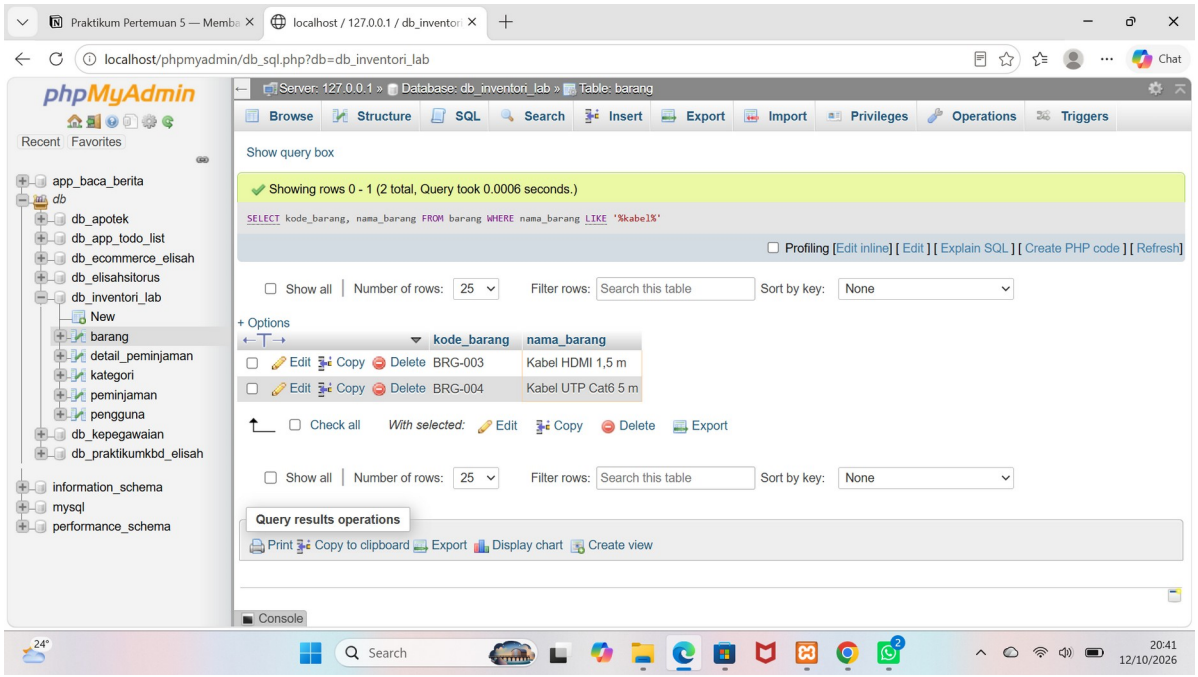
Hasil: Terdapat 3 baris hasil, yaitu Access Point (BRG-006) dengan stok 2, LAN Tester (BRG-008) dengan stok 3, dan Switch 8 Port (BRG-005) dengan stok 4. Ketiga barang ini sebaiknya didahulukan untuk pengadaan atau perbaikan.

5. Pencarian Teks dengan LIKE

Query ini mencari barang yang nama_barang-nya memuat kata "kabel" di bagian mana pun dari teks.

Perintah SQL:

```
SELECT kode_barang, nama_barang
FROM barang
WHERE nama_barang LIKE '%kabel%';
```



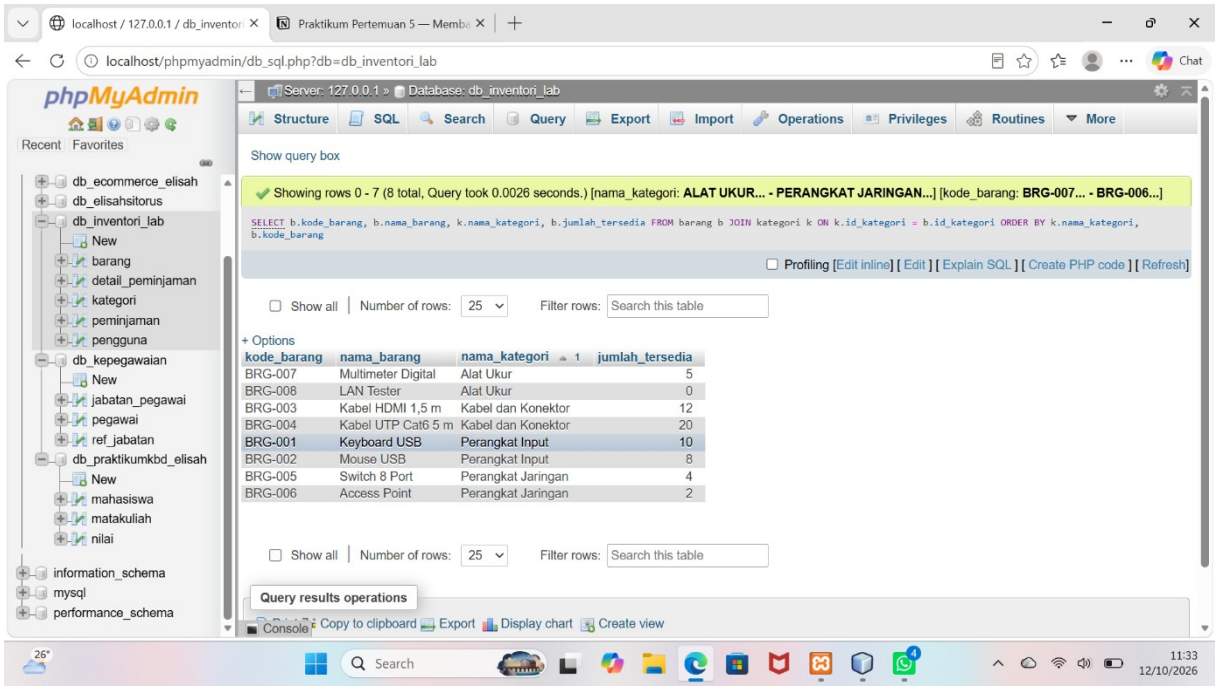
Hasil: Ditemukan 2 baris, yaitu Kabel HDMI 1,5 m (BRG-003) dan Kabel UTP Cat6 5 m (BRG-004).

6. Menggabungkan Dua Tabel dengan JOIN (barang dan kategori)

Tabel barang hanya menyimpan id_kategori berupa angka. Melalui JOIN, id tersebut dipasangkan dengan tabel kategori agar nama kategorinya bisa ditampilkan.

Perintah SQL:

```
SELECT b.kode_barang, b.nama_barang, k.nama_kategori, b.jumlah_tersedia
FROM barang b
JOIN kategori k ON k.id_kategori = b.id_kategori
ORDER BY k.nama_kategori, b.kode_barang;
```



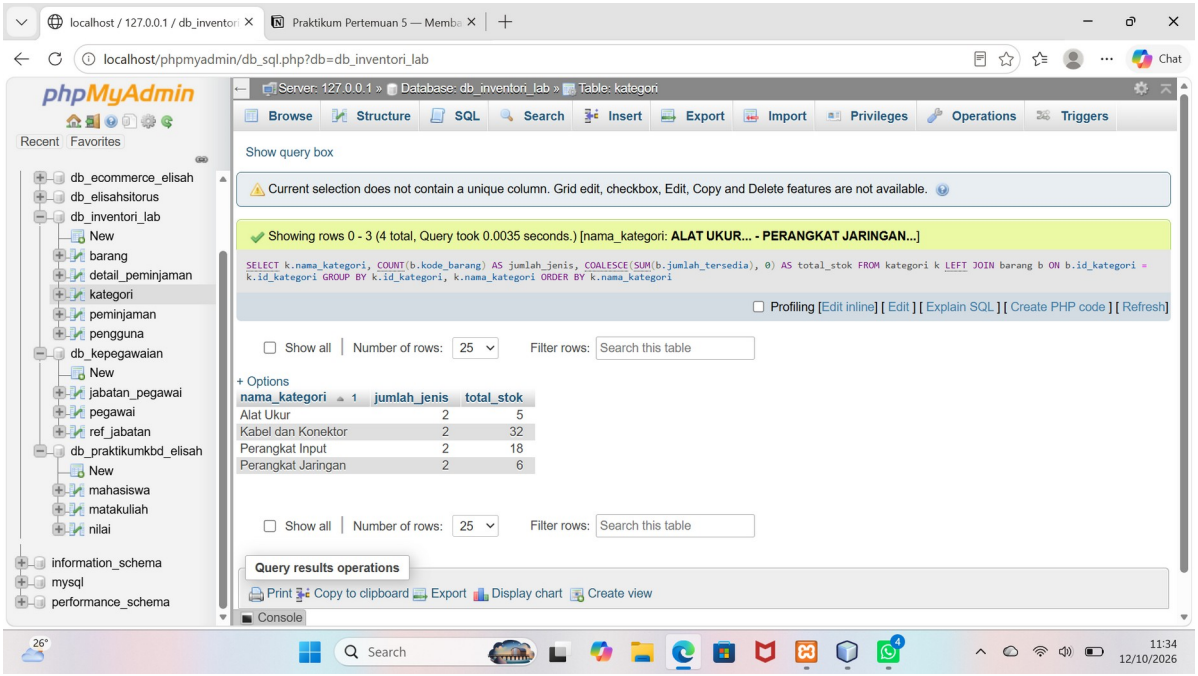
Hasil: Output berupa 8 baris yang sudah menyertakan nama kategori, sehingga lebih mudah dipahami dibanding hanya melihat angka id_kategori.

7. Rekap per Kategori dengan LEFT JOIN, COUNT, SUM, dan GROUP BY

Query ini menyusun rekapitulasi per kategori, yaitu banyaknya jenis barang dan total stok pada masing-masing kategori.

Perintah SQL:

```
SELECT k.nama_kategori,
       COUNT(b.kode_barang) AS jumlah_jenis,
       COALESCE(SUM(b.jumlah_tersedia), 0) AS total_stok
FROM kategori k
LEFT JOIN barang b ON b.id_kategori = k.id_kategori
GROUP BY k.id_kategori, k.nama_kategori
ORDER BY k.nama_kategori;
```



Hasil: Terdapat 4 kategori: Alat Ukur (2 jenis, stok 5), Kabel dan Konektor (2 jenis, stok 32), Perangkat Input (2 jenis, stok 18), dan Perangkat Jaringan (2 jenis, stok 6).

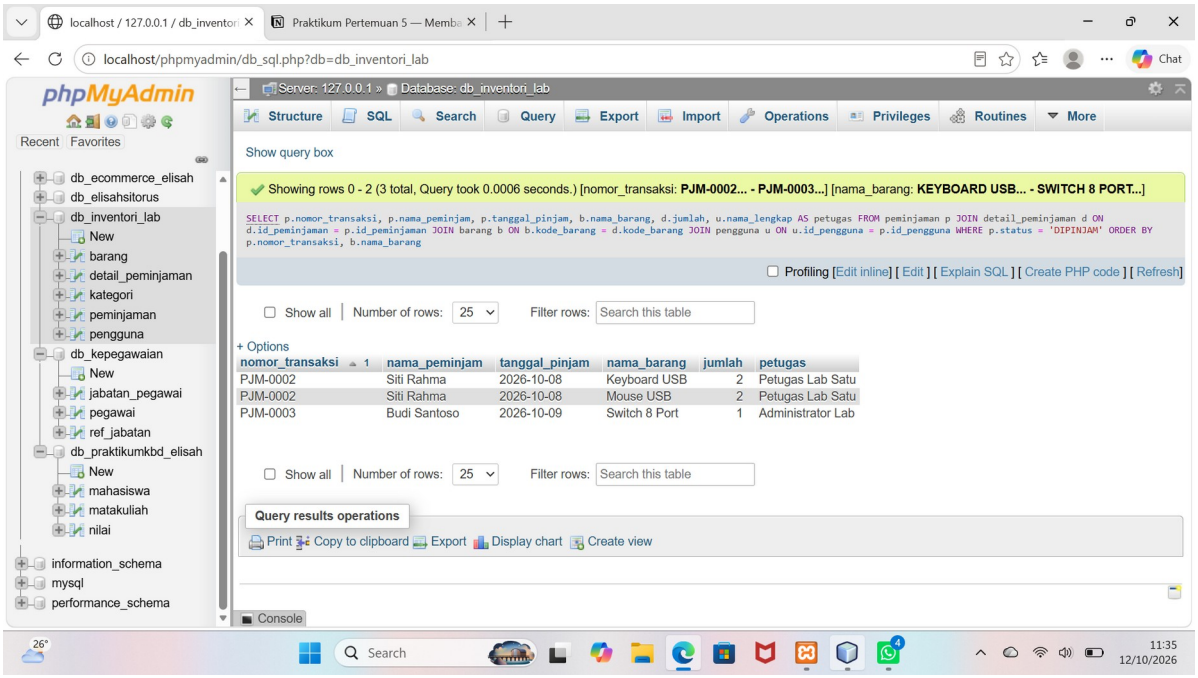
8. Laporan Peminjaman Aktif (JOIN Empat Tabel)

Empat tabel (peminjaman, detail_peminjaman, barang, dan pengguna) digabungkan dalam satu query untuk menghasilkan laporan peminjaman yang masih berstatus dipinjam.

Perintah SQL:

```
SELECT p.nomor_transaksi, p.nama_peminjam, p.tanggal_pinjam,
       b.nama_barang, d.jumlah, u.nama_lengkap AS petugas
FROM peminjaman p
```

```
JOIN detail_peminjaman d ON d.id_peminjaman = p.id_peminjaman
JOIN barang b ON b.kode_barang = d.kode_barang
JOIN pengguna u ON u.id_pengguna = p.id_pengguna
WHERE p.status = 'DIPINJAM'
ORDER BY p.nomor_transaksi, b.nama_barang;
```



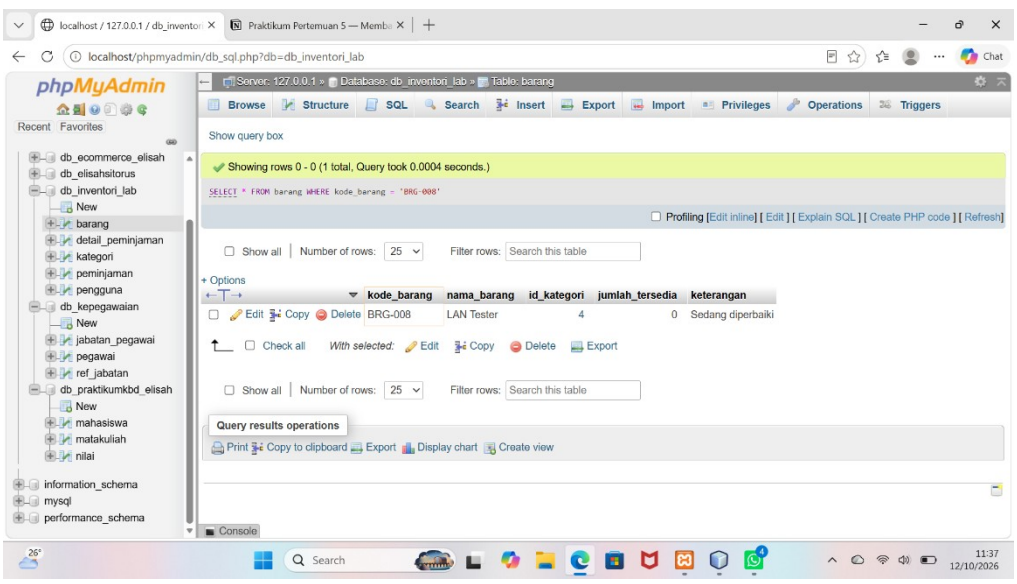
Hasil: Output terdiri dari 3 baris. Transaksi PJM-0002 atas nama Siti Rahma (2026-10-08) mencatat peminjaman Keyboard USB 2 unit dan Mouse USB 2 unit yang dilayani Petugas Lab Satu, sedangkan transaksi PJM-0003 atas nama Budi Santoso (2026-10-09) mencatat peminjaman Switch 8 Port 1 unit yang dilayani Administrator Lab.

9. Memeriksa Data Sebelum UPDATE

Sebelum data diubah, baris target dilihat lebih dulu untuk memastikan perubahan tidak mengenai data yang salah.

Perintah SQL:

```
SELECT * FROM barang
WHERE kode_barang = 'BRG-008';
```



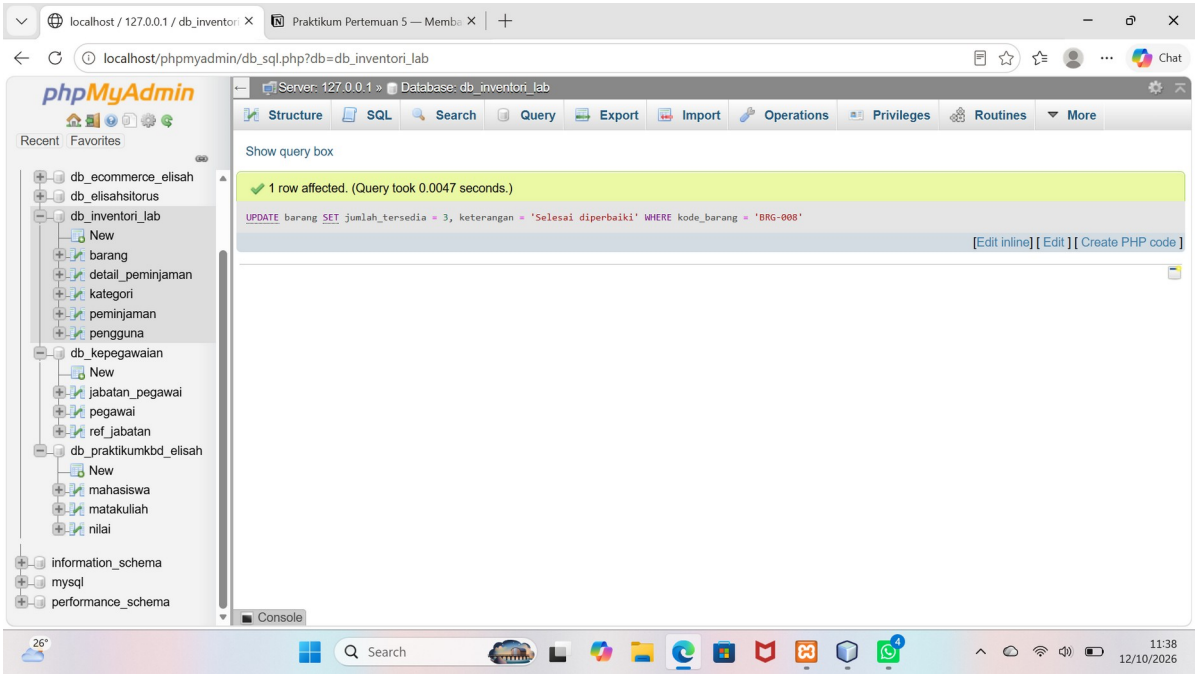
Hasil: Kondisi awal ini menjadi acuan untuk membandingkan data sebelum dan sesudah UPDATE dijalankan.

10. Mengubah Data dengan UPDATE

UPDATE digunakan untuk mengubah data yang sudah ada. Pada contoh ini, LAN Tester yang sudah selesai diperbaiki dicatat kembali sebagai barang yang tersedia.

Perintah SQL:

```
UPDATE barang
SET jumlah_tersedia = 3,
    keterangan = 'Selesai diperbaiki'
WHERE kode_barang = 'BRG-008';
```



Hasil: Data BRG-008 berhasil diubah, dan perubahannya dapat dilihat pada tampilan data barang di bagian 2.

11. Menambah Data dengan INSERT dan Menghapus dengan DELETE

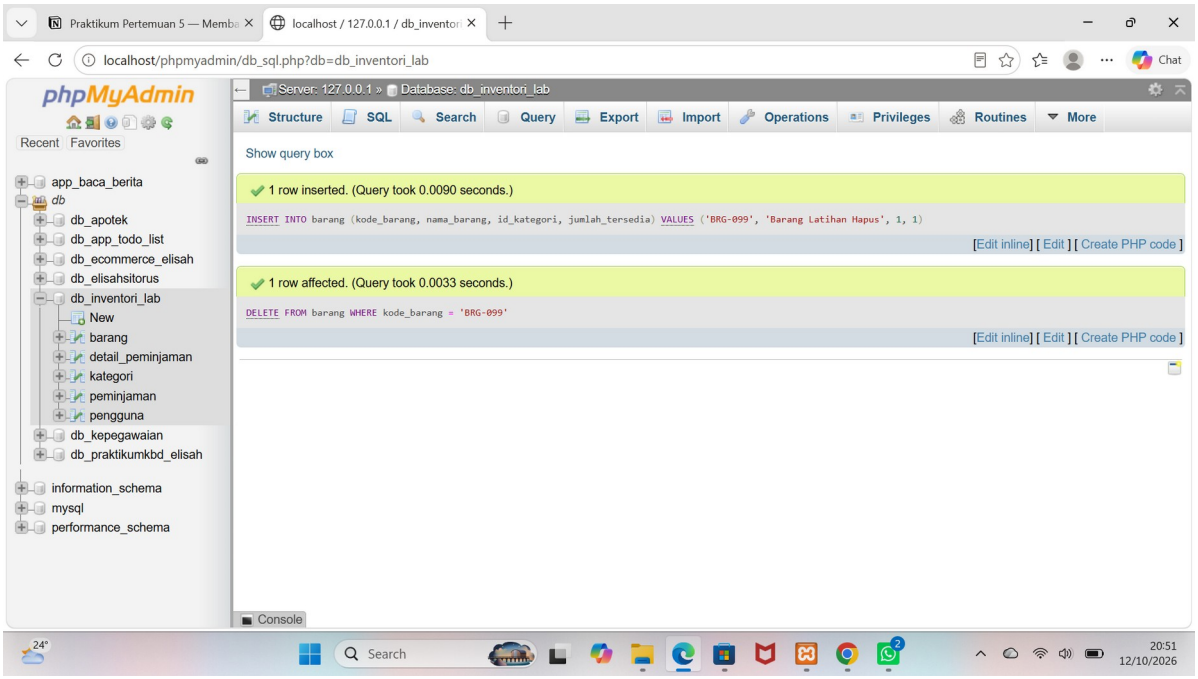
Dua perintah dieksekusi secara berurutan: satu barang percobaan ditambahkan, kemudian langsung dihapus kembali.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-099', 'Barang Latihan Hapus', 1, 1);

DELETE FROM barang
```

```
WHERE kode_barang = 'BRG-099';
```



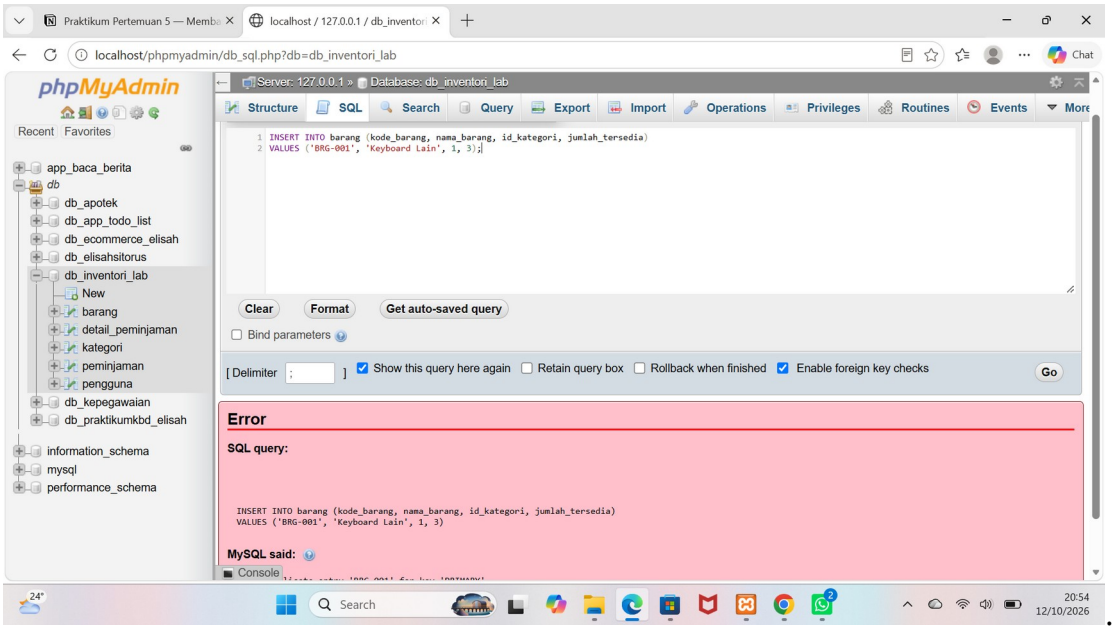
Hasil: Barang percobaan berhasil ditambahkan dan dihapus, sehingga isi tabel barang kembali seperti sebelumnya.

12. Uji Error 1: Primary Key Duplikat

Percobaan ini sengaja memasukkan kode_barang yang sudah ada guna menguji bahwa primary key berfungsi.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-001', 'Keyboard Lain', 1, 3);
```



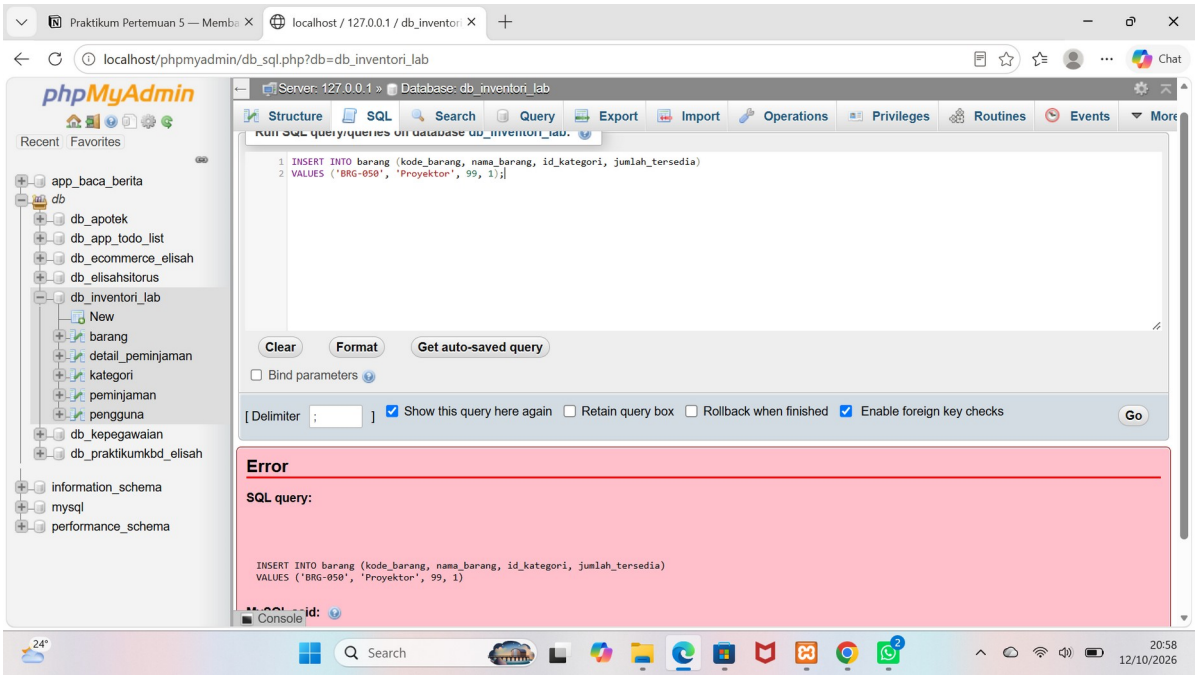
Hasil: Query ditolak dan data tetap utuh. Hal ini menunjukkan bahwa primary key melarang adanya dua barang dengan kode yang sama.

13. Uji Error 2: Pelanggaran Foreign Key (Kategori Tidak Ada)

Percobaan ini menambahkan barang dengan id_kategori 99, padahal kategori tersebut tidak terdaftar di tabel kategori.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-050', 'Proyektor', 99, 1);
```



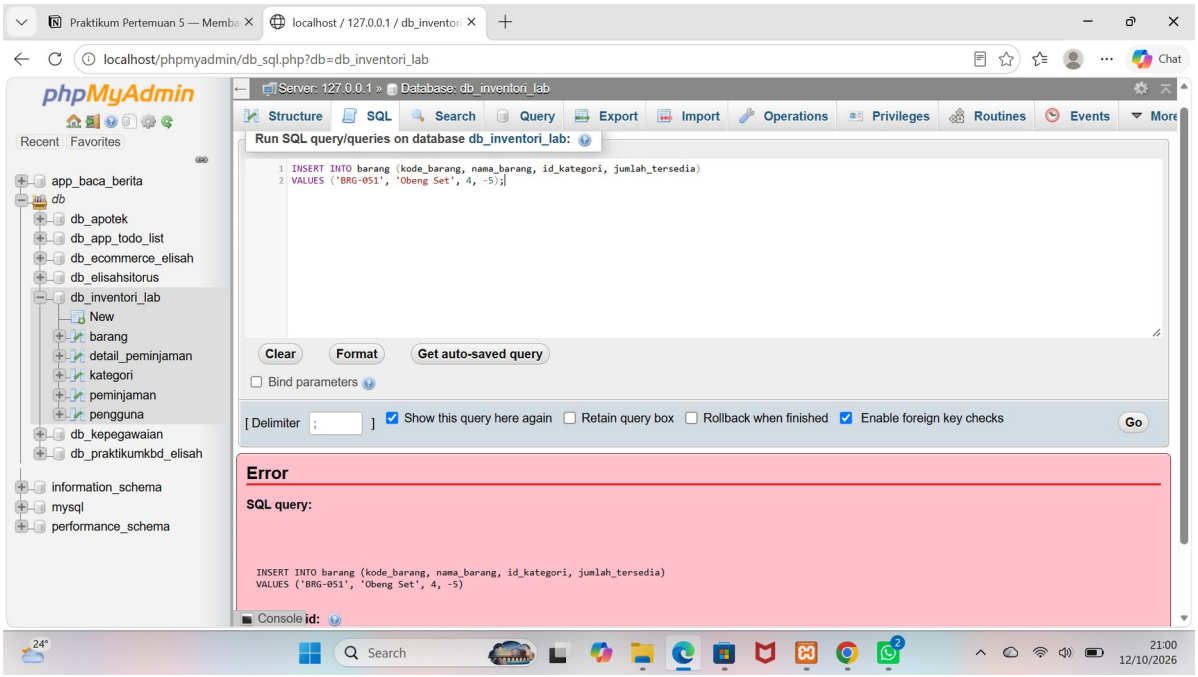
Hasil: Query ditolak. Foreign key menjaga integritas referensial sehingga setiap barang selalu terkait dengan kategori yang valid.

14. Uji Error 3: Pelanggaran CHECK Constraint (Stok Negatif)

Percobaan ini mengisi stok dengan nilai negatif (-5), yang tidak masuk akal untuk data persediaan.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-051', 'Obeng Set', 4, -5);
```



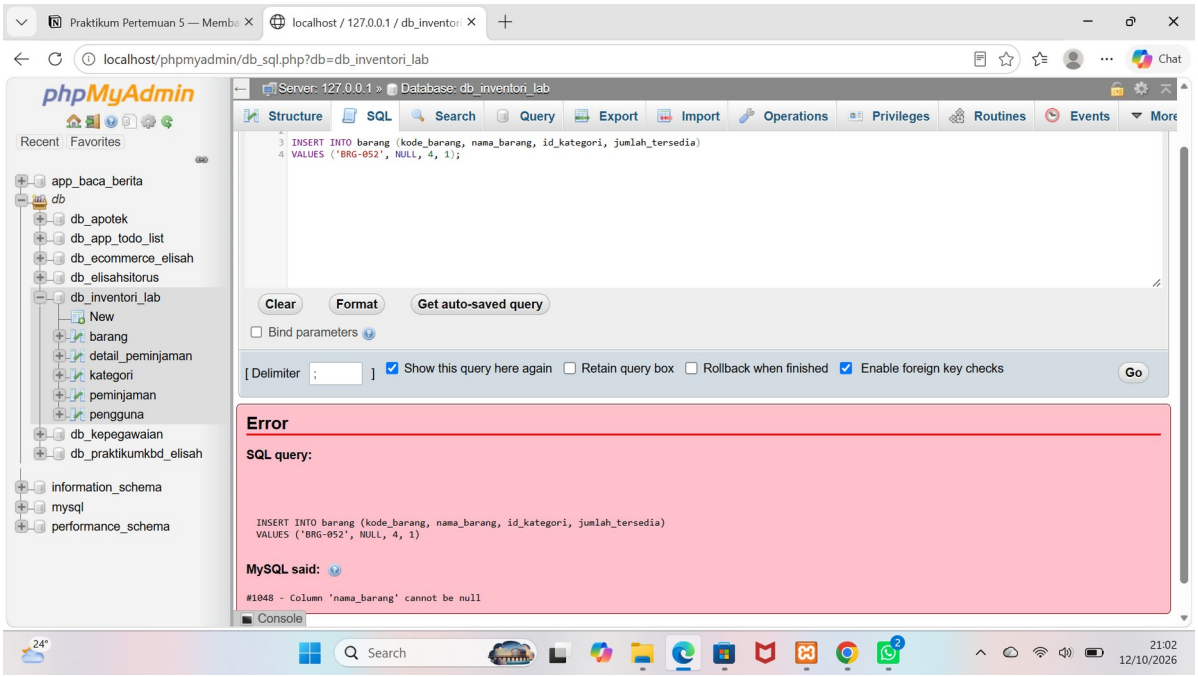
Hasil: Query ditolak. CHECK constraint menjamin nilai stok selalu berada pada batas yang wajar.

15. Uji Error 4: Kolom NOT NULL Diisi NULL

Percobaan ini mengosongkan nama_barang dengan mengisinya NULL.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-052', NULL, 4, 1);
```



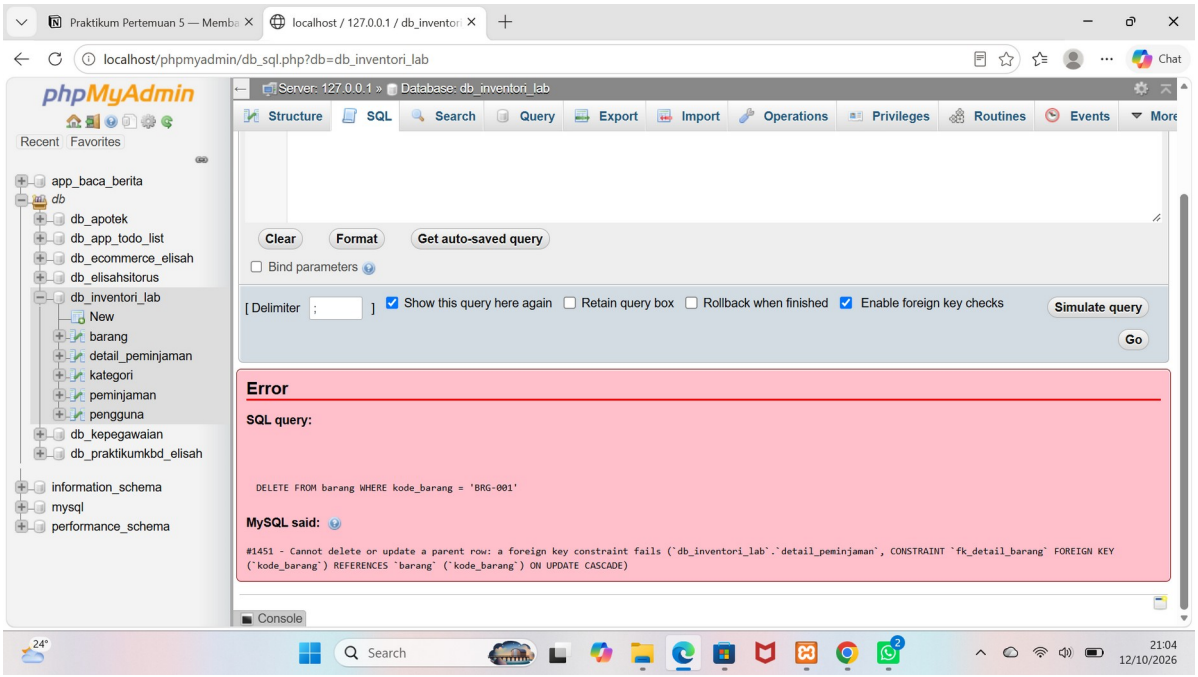
Hasil: Query ditolak. Aturan NOT NULL melarang barang disimpan tanpa nama.

16. Uji Error 5: Menghapus Barang yang Masih Dipakai (Foreign Key)

Percobaan ini berusaha menghapus Keyboard USB (BRG-001) yang sudah muncul pada transaksi peminjaman.

Perintah SQL:

```
DELETE FROM barang
WHERE kode_barang = 'BRG-001';
```



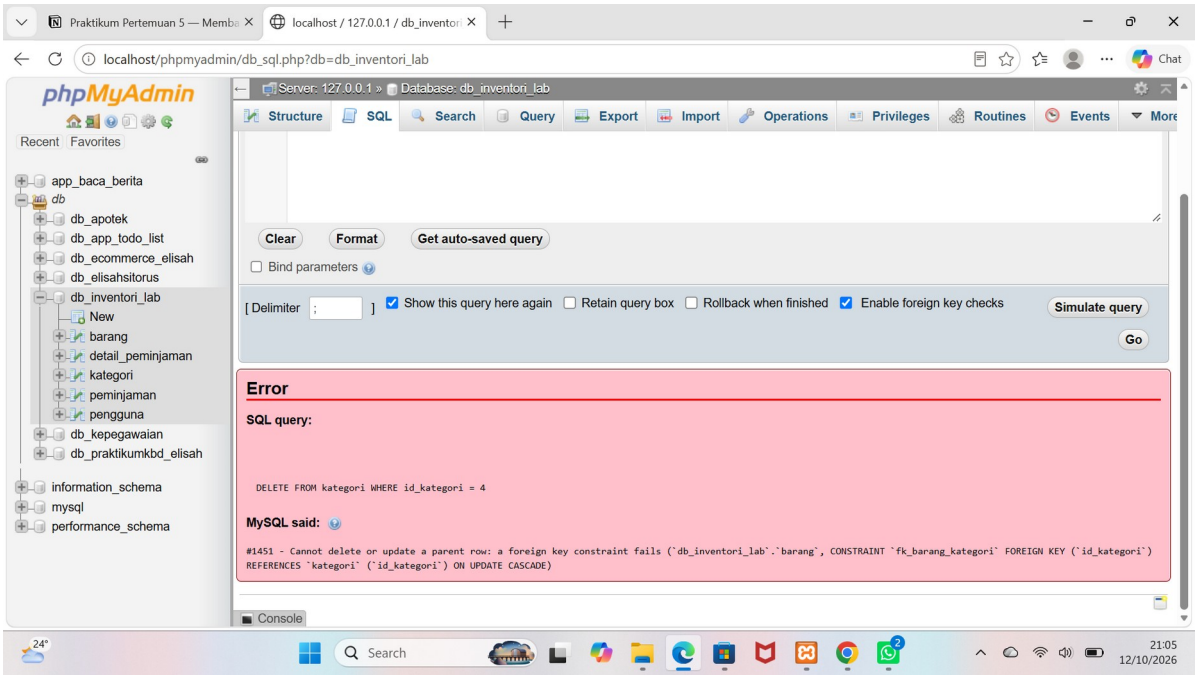
Hasil: Query ditolak. Dengan begitu, data barang yang dirujuk oleh riwayat peminjaman tidak ikut hilang.

17. Uji Error 6: Menghapus Kategori yang Masih Dipakai Barang

Percobaan ini berusaha menghapus kategori dengan id 4 (Alat Ukur) yang masih digunakan oleh tabel barang.

Perintah SQL:

```
DELETE FROM kategori
WHERE id_kategori = 4;
```



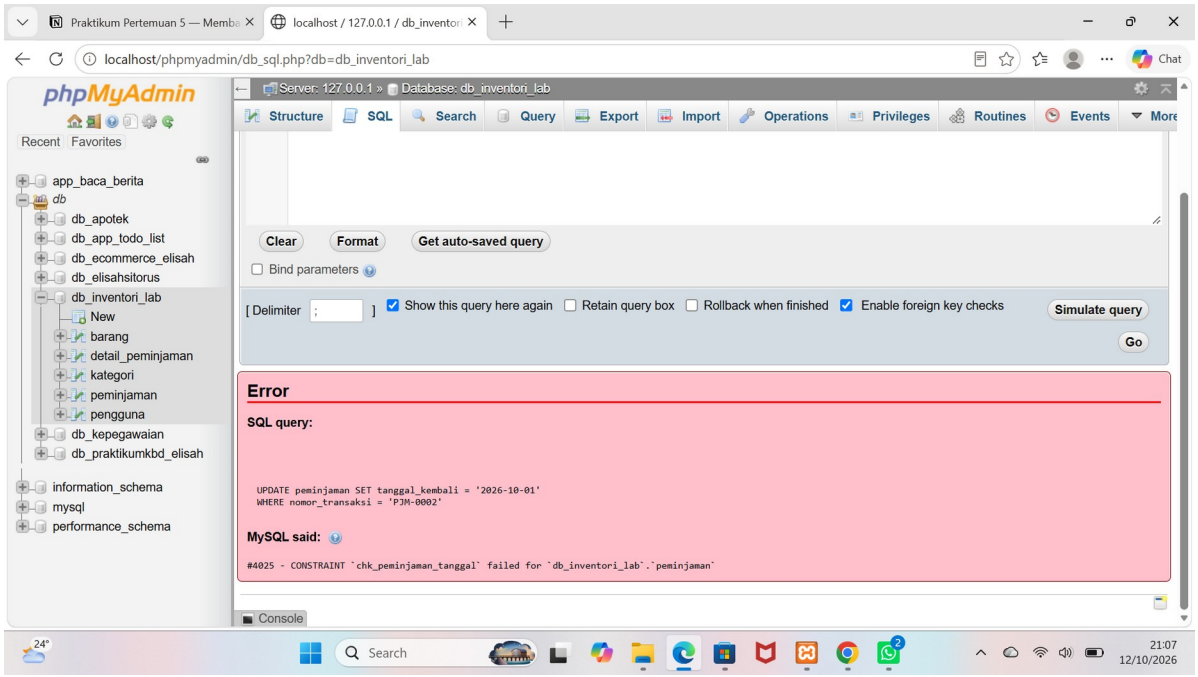
Hasil: Query ditolak. Supaya kategori bisa dihapus, barang yang memakainya perlu dihapus atau dipindahkan ke kategori lain lebih dahulu.

18. Uji Error 7: Pelanggaran CHECK pada Tanggal Peminjaman

Percobaan ini mengisi tanggal kembali (2026-10-01) yang lebih awal daripada tanggal pinjam transaksi PJM-0002 (2026-10-08).

Perintah SQL:

```
UPDATE peminjaman
SET tanggal_kembali = '2026-10-01'
WHERE nomor_transaksi = 'PJM-0002';
```



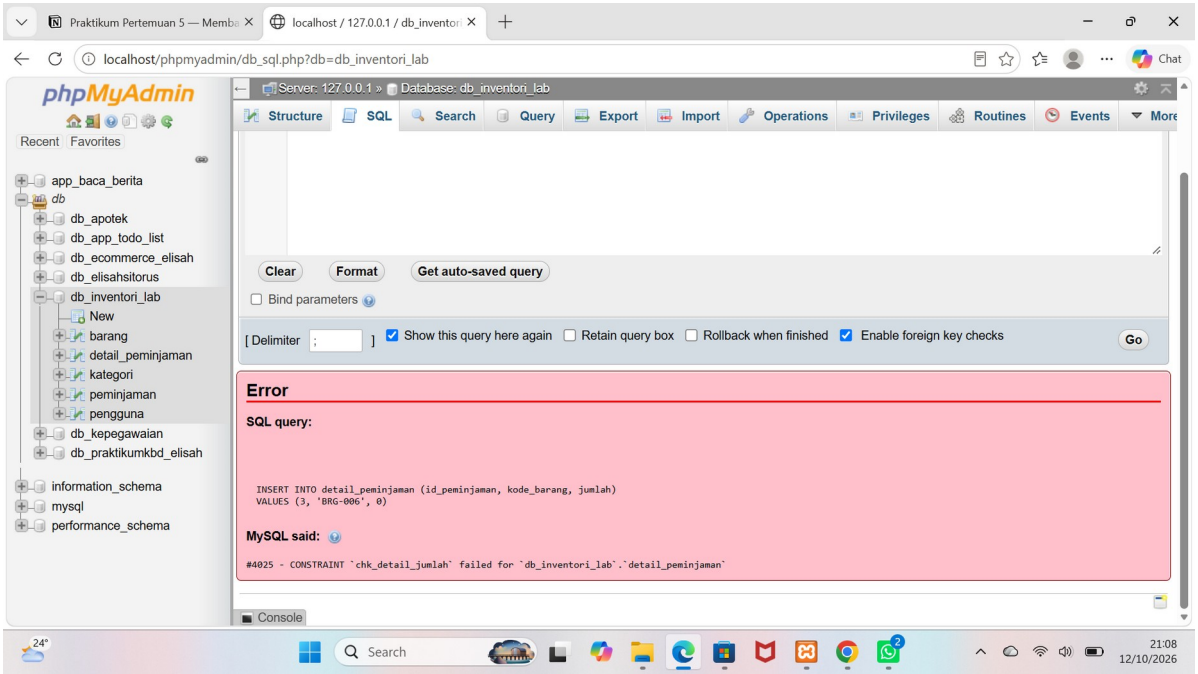
Hasil: Query ditolak, sehingga data tanggal peminjaman tetap masuk akal.

19. Uji Error 8: Pelanggaran CHECK pada Jumlah Detail Peminjaman

Percobaan ini mencatat peminjaman barang dengan jumlah 0.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (3, 'BRG-006', 0);
```



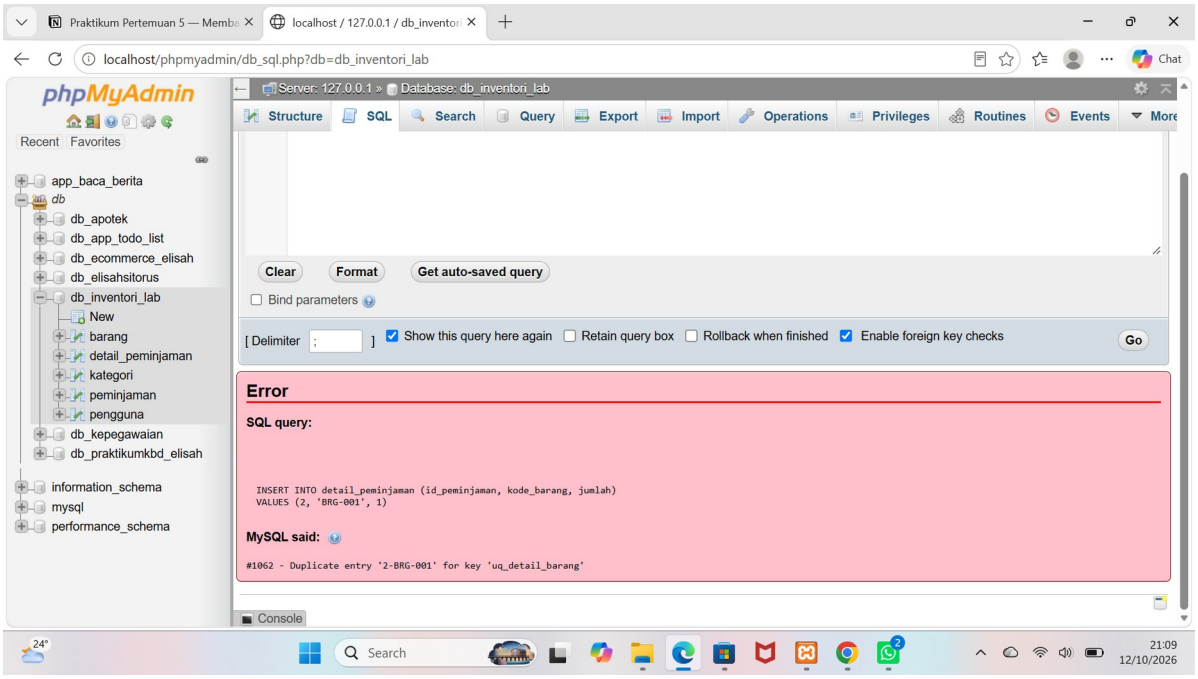
Hasil: Query ditolak, karena peminjaman dengan jumlah barang nol tidak masuk akal.

20. Uji Error 9: Unique Key Ganda pada Detail Peminjaman

Percobaan ini mencatat BRG-001 (Keyboard USB) pada transaksi dengan id_peminjaman 2 (PJM-0002), padahal barang itu sudah tercatat di transaksi tersebut.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (2, 'BRG-001', 1);
```



Hasil: Query ditolak. Unique key mencegah munculnya baris detail kembar untuk barang yang sama dalam satu transaksi.

21. Menambah Data Peminjaman Latihan beserta Detailnya

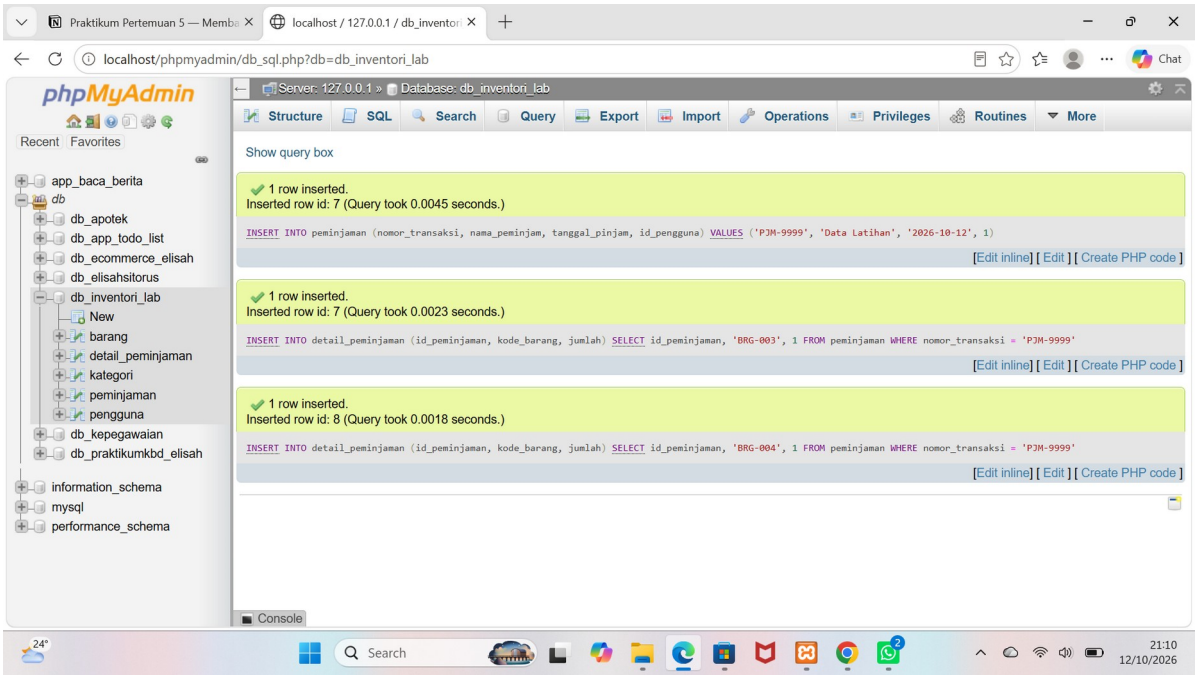
Tiga perintah INSERT dijalankan berurutan untuk membentuk satu transaksi percobaan (PJM-9999) yang berisi dua barang pada tabel detail.

Perintah SQL:

```
INSERT INTO peminjaman (nomor_transaksi, nama_peminjam, tanggal_pinjam,
id_pengguna)
VALUES ('PJM-9999', 'Data Latihan', '2026-10-12', 1);

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-003', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-004', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';
```



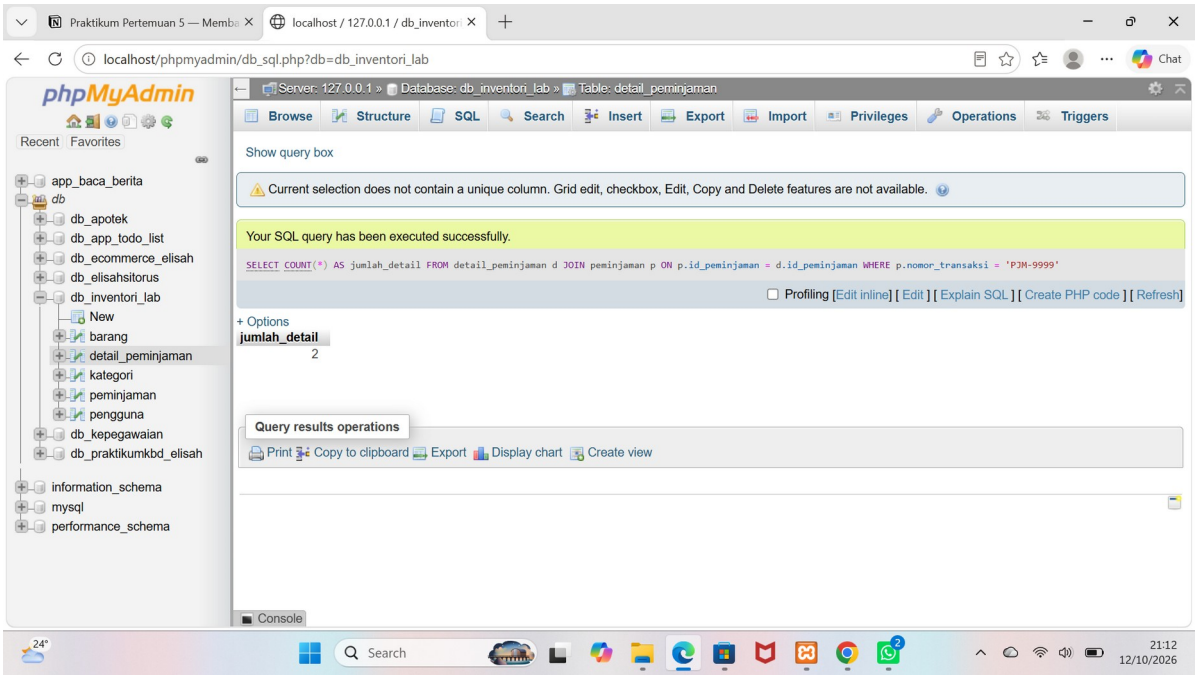
Hasil: Ketiga perintah sukses dijalankan (masing-masing 1 row inserted). Data ini disiapkan untuk menguji penghapusan berantai pada langkah selanjutnya.

22. Menghitung Jumlah Detail Transaksi PJM-9999

Query ini memverifikasi bahwa transaksi PJM-9999 memang memiliki baris detail sebelum transaksinya dihapus.

Perintah SQL:

```
SELECT COUNT(*) AS jumlah_detail
FROM detail_peminjaman d
JOIN peminjaman p ON p.id_peminjaman = d.id_peminjaman
WHERE p.nomor_transaksi = 'PJM-9999';
```



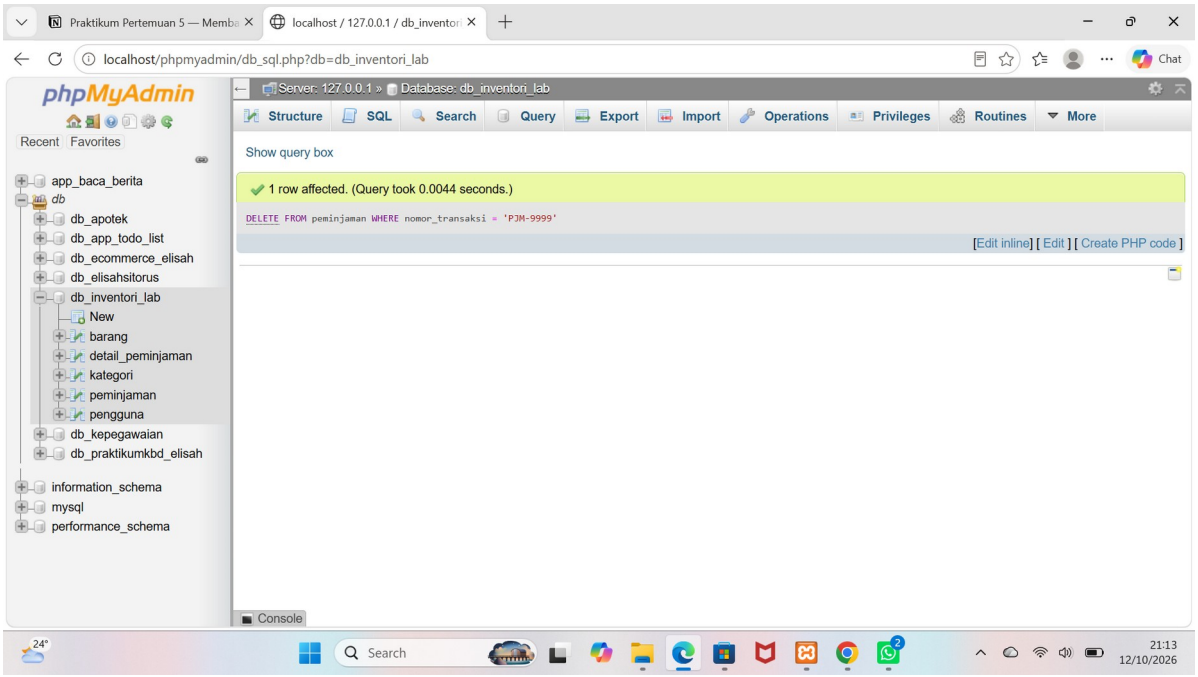
Hasil: Hasilnya menunjukkan ada 2 baris detail untuk PJM-9999, yang menjadi kondisi awal sebelum DELETE.

23. Menghapus Transaksi Peminjaman (DELETE pada Tabel Induk)

Transaksi percobaan PJM-9999 dihapus dari tabel peminjaman, walaupun masih memiliki 2 baris detail di tabel anak.

Perintah SQL:

```
DELETE FROM peminjaman
WHERE nomor_transaksi = 'PJM-9999';
```



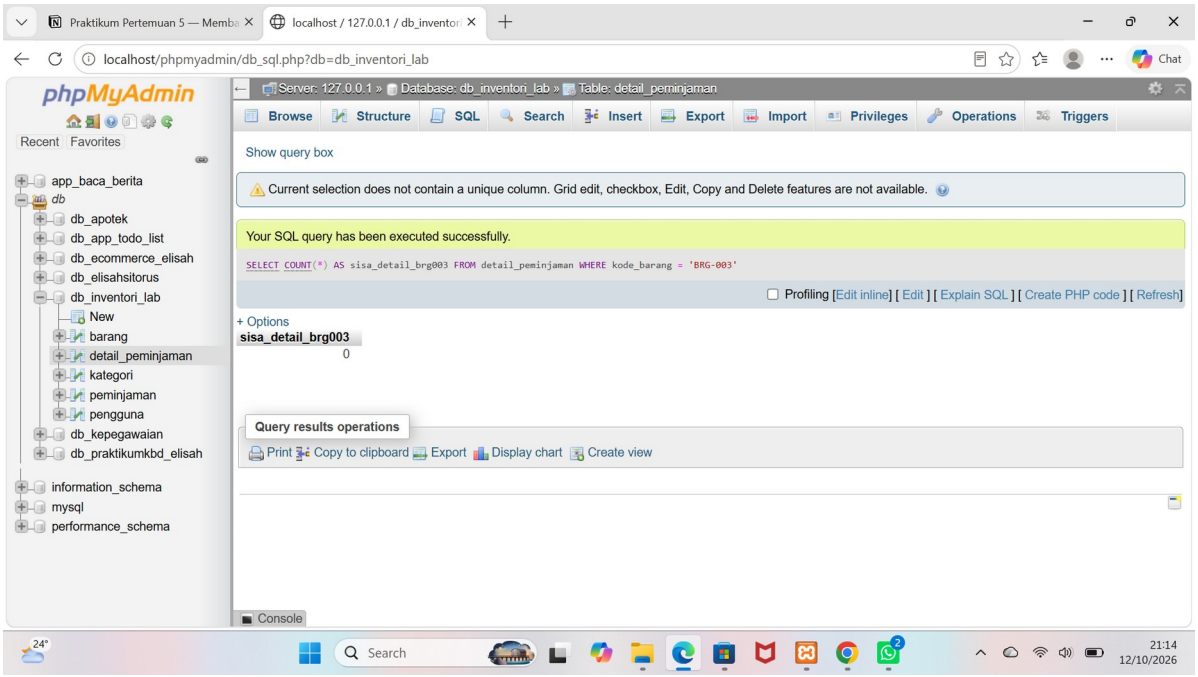
Hasil: Transaksi PJM-9999 berhasil dihapus. Bukti bahwa baris detailnya ikut terhapus ditunjukkan pada langkah berikutnya.

24. Pembuktian Penghapusan Berantai (CASCADE) pada Detail

Query ini memeriksa apakah masih ada baris detail milik PJM-9999 setelah transaksi induknya dihapus. BRG-003 dipilih karena barang ini hanya muncul pada transaksi PJM-9999.

Perintah SQL:

```
SELECT COUNT(*) AS sisa_detail_brg003
FROM detail_peminjaman
WHERE kode_barang = 'BRG-003';
```



Hasil: Penghapusan berantai terbukti berjalan: saat data induk di tabel peminjaman dihapus, data anak di tabel detail_peminjaman ikut terhapus secara otomatis.